ABAP INTERVIEW PREPARATION

Senior ABAP Cloud mock interview

Six connected cases, code diagnosis, model reasoning and a 75-minute self-check

350 questions across 25 study segments. Reviewed 13 October 2026.

Use the short answers for oral rehearsal. The examples and pitfalls prepare you for follow-up questions. Choose the target SAP edition and release before applying an implementation detail.

Open abap-interview-studio.pages.dev online, or download the repository and open START_HERE.html for offline study. The PDF is a portable reading and printing edition.

This 75-minute interview follows one purchase-approval extension from architecture to a production incident. It tests decisions and failure reasoning rather than memorized definitions. The scenarios are original practice cases. Establish the target edition, ABAP release, language version and consumer before promising support.

All five code blocks are intentionally incomplete teaching fragments, some deliberately defective. Names are hypothetical. They are not compilable repository objects or verified executable APIs. Validate real syntax, types, API release state and runtime behavior in the target SAP system before implementation.

Interview schedule

Segment	Minutes	Cumulative
Read context and clarify assumptions	3	3
Exercise 1: architecture and Clean Core	11	14
Exercise 2: partial updates and rejection	12	26
Exercise 3: security across layers	11	37
Exercise 4: custom query correctness	10	47
Exercise 5: posting and transaction ownership	11	58
Exercise 6: concurrency and recovery	12	70
Review evidence gaps and score	5	75

Shared context: employees create purchase requests with items. Approvers are restricted by company code and approval limit. A supplier portal reads approved requests. Posting creates an SAP purchasing document. Requests support draft editing. One integration dependency may be unavailable, and users may retry operations after timeouts.

Scoring rubric

Score four dimensions from 0 to 3 for each exercise: boundary, mechanism, trade-off and evidence. The maximum is 12 per exercise and 72 overall. These are self-assessment criteria, not an employer's hiring scale.

Score	Boundary	Mechanism	Trade-off	Evidence
0	Misses transaction or trust boundary	Incorrect or absent mechanism	No alternatives considered	Claims success without verification
1	Names a boundary without tracing it	Names a relevant feature	Names a cost without deciding	Proposes only a happy-path test
2	Explains ownership and failure path	Applies the right supported contract	Defends a choice against an alternative	Defines observable negative tests
3	Handles cross-layer and retry effects	Qualifies release limits and exceptions	Links decision to business consequences	Specifies fault injection, assertions and operational evidence

Do not average away a dangerous misconception. Independent commits inside RAP behavior, authorization bypass, silently ignored filters or assumed remote atomicity require correction even if the total is high. For readiness, aim for at least 2 in every dimension on every case, then repeat weak cases aloud without reading the model answer.

Exercise 1 Design the extension and defend its Clean Core position

The architect proposes one on-stack RAP BO containing request, items, supplier portal sessions and the eventual standard purchase order. A developer says it is automatically Clean Core Level A because RAP is used. The only familiar posting function is not released for ABAP Cloud consumption.

Explain the BO boundary, deployment placement and dependency decision. Identify which missing facts could change your design, and give the sponsor a practical path forward.

Your answer before reading the solution

Model answer

Keep the request and its items in one BO when they need to be edited, locked and saved together. The standard purchase order is another business object, so access it through an appropriate API. Portal sign-in sessions do not belong inside the purchase request. For new custom tables, managed RAP is usually a useful starting point.

Choose where the extension runs by asking what must save together, which APIs exist and whether the portal needs its own release schedule. RAP alone does not make a solution Clean Core Level A. Check the APIs and extension techniques it actually uses. Prefer a released posting operation. A wrapper does not remove the risk of an internal dependency. A classic exception may be possible in a suitable edition, but needs an owner, upgrade tests, an expiry and a replacement plan; it is not general ABAP Cloud permission.

Probes

What requirement genuinely needs the same local transaction? What changes if only a remote posting API exists? How would you justify a temporary dependency exception without calling it Level A?

Wrong answers to challenge

"RAP makes every dependency cloud compliant." "A side-by-side application has no core dependencies." "A wrapper converts any internal object into a released SAP API."

Evidence and scoring focus

Draw trust and transaction boundaries. Produce a dependency inventory with target API states, one architecture decision and an upgrade test. Score the distinction between placement, language restrictions and Clean Core classification.

Study links: CLOUD-01, CLOUD-05 and CLOUD-07 in cloud architecture; GOV-01, GOV-03 and GOV-07 in Clean Core governance.

Exercise 2 Diagnose a partial update and a validation that does not reject

A caller clears an approver comment, but its previous text survives. Another request receives an error message yet still saves an invalid amount. Diagnose both fragments and explain how a bulk request containing valid and invalid items should be tested.

```
" Defective teaching fragment: context and types omitted.
" The caller intends to clear Comment for an existing request.
ls_update-Comment = ''.
ls_update-%control-Comment = if_abap_behv=>mk-off.
" ls_update is subsequently passed to EML UPDATE FROM.

" Defective validation fragment: generated signature omitted.
" ls_reported contains an error message and the instance identity.
APPEND ls_reported TO reported-Request.
" The rejected instance is never appended to failed-Request.
```

Your answer before reading the solution

Model answer

An empty Comment value means 'clear it' only when the update also marks Comment as supplied. The first fragment uses mk-off, so the caller is not asking RAP to change that field. Mark only the intended fields. Marking every field could overwrite an omitted Amount with zero. Check adapters and defaulting code too if they might turn an explicit clear back into old text.

In the validation, add the rejected instance to FAILED and its message to REPORTED. Include the right RAP identifier, field and path where required. A determination cannot fill FAILED because it has a different response signature. For a bulk request, process every item and link each error to its item. Then check which operations the caller and protocol group to succeed or fail together; one FAILED row does not automatically roll back an entire OData batch.

Probes

How do omitted Amount and Amount equal to zero differ? Which identity components matter for drafts? Does one FAILED entry always roll back every operation in an entire OData batch?

Wrong answers to challenge

"IS INITIAL means the field was omitted." "An error-severity message always stops saving." "Only the first failed item matters."

Evidence and scoring focus

Test omitted, empty and nonempty Comment; confirm untouched fields survive. Submit several items with two different validation failures and assert their identities, messages and final persistence. Distinguish interaction responses from save outcome.

Study links: SRAP-05, SRAP-06, SRAP-07 and SRAP-20 in RAP transaction scenarios.

Exercise 3 Repair read and operation authorization together

The base request view has DCL. A new service projection exposes requests from other company codes. The UI hides Approve for ordinary users, but a direct action request still succeeds. A third user can update a permitted request so that its CompanyCode becomes an unauthorized one.

```
// Incomplete CDS teaching fragment, not a complete model.
@AccessControl.authorizationCheck: #CHECK
define view entity ZC_Request
  as projection on ZI_Request
{
  key RequestId,
      CompanyCode,
      Amount
}
// DCL is known only for ZI_Request; none is shown for ZC_Request.
```

Your answer before reading the solution

Model answer

Start with the entity the service actually reads. #CHECK asks for access-control checks, but does not create DCL or automatically copy the base entity's rules. Define the projection's rules or use supported explicit inheritance with the right field mappings. Test reads, navigation and value helps through the real service.

Approve needs its own backend authorization check. Hiding its button is only a UI aid. Feature control separately describes whether the request's current state allows approval. When CompanyCode changes from A to B, check whether the user may make that proposed change; permission for the original A record is not enough. Inspect the supplied new values through supported mechanisms, and keep the distinction between an omitted field and a field explicitly changed to an empty value.

Probes

Can DCL alone authorize Approve? What checks can IN LOCAL MODE bypass, and where is that mode legitimate? How would you prevent information leakage through value helps or counts?

Wrong answers to challenge

"#CHECK creates authorization automatically." "Hidden buttons protect the API." "Any authorized update may move data to any company code."

Evidence and scoring focus

Use permitted and denied users with known foreign keys. Test reads, direct actions and A-to-B company changes. Compare UI behavior with backend rejection and ensure negative tests assert absence of data, not merely an error banner.

Study links: MODEL-01, MODEL-02, MODEL-11 and MODEL-13 in CDS and security; SRAP-12 in RAP scenarios.

Exercise 4 Make a remote custom query correct before optimizing it

The supplier portal reads a custom entity. A complex filter produces an exception; the developer catches it and returns everything. The count is the number of rows on the current page, and no stable sort is applied.

```
DELIBERATELY DEFECTIVE PSEUDOCODE; NOT ABAP
try: filter = convert_request_to_simple_ranges()
catch unsupported_filter: filter = no_filter
rows = fetch_remote_rows(filter)
page = first_requested_slice(rows)
count = length(page)
return page, count
```

Your answer before reading the solution

Model answer

Do not return all records when a filter cannot be converted. Either process the complete filter correctly with a supported method or return a clear error saying it is unsupported. Apply the same filtering and permissions to both data and count. If fifty allowed rows match and the page contains ten, the total is fifty. Only calculate a count when the caller requests it.

Sort with a reliable tie-breaker, such as a suitable unique key, before taking a page. Ask the remote provider to filter, sort and page when it supports those operations. Sorting alone does not freeze data between requests: new rows can still arrive. Check the remote API's continuation or snapshot rules before promising stable pages. Fetching every record just to discard most locally can be slow and expensive.

Probes

What happens with an OR expression that simple ranges cannot represent? How would you process count-only requests? What changes when records are inserted between pages?

Wrong answers to challenge

"Returning more rows is a harmless fallback." "Count equals page size." "ORDER BY any nonunique field prevents all paging anomalies."

Evidence and scoring focus

Use complex filters, no matches, count-only, second-page and concurrent-insert cases. Assert authorization and total count. Measure remote call count and transferred rows under representative volume.

Study links: MODEL-03, MODEL-04 and MODEL-05 in CDS and security; OPS-07 and OPS-08 in production readiness.

Exercise 5 Integrate posting without losing transaction ownership

The local prototype reaches a nested COMMIT WORK and violates the RAP runtime contract. A proposed RFC workaround commits a purchasing document remotely, but the local request later fails validation. The developer proposes moving posting to SAVE.

```
" Defective local prototype: forbidden nested commit.
" A posting API was called from a RAP behavior implementation.
" Deeper in that call chain, legacy code executes:
COMMIT WORK.
" The RAP request has not completed its own save decision.
```

Your answer before reading the solution

Model answer

The local nested COMMIT WORK violates RAP's rules. Do not describe it as a normal commit followed by continued validation. Moving that same call into SAVE, or wrapping it, does not remove its internal commit. Find an API that fits RAP's save rules and check the whole call chain.

A remote call has a different problem: it can commit in the other system before the local request fails. Local rollback cannot undo it. If one local all-or-nothing save is not supported, store a posting request and process it separately, with a way to check uncertain outcomes and prevent duplicates. Check predictable business errors early. Some releases support special failure-capable savers for specific BAPI cases, including late numbering. Verify that exact scenario; it does not permit arbitrary self-committing legacy code.

Probes

What if the final document number appears only after posting? What if the remote call times out after succeeding? Which state allows operations staff to reconcile an uncertain result safely?

Wrong answers to challenge

"Unmanaged save permits any COMMIT." "An RFC wrapper restores atomicity." "Retry every timeout as a new create."

Evidence and scoring focus

Trace the first durable effect. Inject failures before and after posting, confirm numbering correlation, and replay an uncertain result. Explain how duplicate documents are prevented and how support establishes the actual outcome.

Study links: SRAP-03, SRAP-04, SRAP-08 and SRAP-15 in RAP scenarios; OPS-11 in production readiness.

Exercise 6 Resolve a stale draft and duplicate approval event

A user leaves an edit draft open. Its exclusive lock expires. Another process changes an active item. The user resumes and submits; two nearly simultaneous Submit requests arrive. The approval event is delivered twice, while the UI still shows the previous total.

Separate the failures. Propose the smallest coherent corrections and a release gate that would detect their recurrence.

Your answer before reading the solution

Model answer

Check the draft's total ETag first. It must reflect relevant active changes, including changes to an item, so resuming an outdated draft can be rejected after its lock expires. The entity ETag used for an ordinary update and the total ETag used for draft resume are related but different checks.

Make two concurrent Submit calls produce only one effective submission. Define what the second caller receives if the request is already submitted, and durably record the follow-up work. The event consumer must also recognize duplicate delivery using a stable identifier. A lost reply can leave the outcome unknown, so check saved state before repeating work. Diagnose the stale total separately: verify the backend value, then inspect side-effect metadata and UI refresh requests. Refreshing the screen must not introduce another commit.

Probes

How do you test genuine concurrency with two sessions? What if a cancellation occurs before a delayed approval event arrives? Which evidence survives a business rollback without leaking sensitive data?

Wrong answers to challenge

"A lock protects a draft forever." "Events are always delivered exactly once." "Refreshing the whole page proves the backend calculation is correct."

Evidence and scoring focus

Run child-only change after lock expiry, concurrent Submit, response loss, duplicate and stale event delivery, and UI refresh checks. Assert one effective transition and intended external effect. The release gate includes backend negative tests, actual consumer behavior, monitoring, deployment prerequisites and recovery ownership; passing ATC alone is insufficient.

Study links: SRAP-10, SRAP-18, SRAP-21 and SRAP-23 in RAP scenarios; OPS-05, OPS-06 and OPS-14 in production readiness.

Five-minute debrief

Record each exercise's four scores and the weakest claim you made. Replace vague phrases such as "the framework handles it" with the named owner, relevant contract and observable assertion. Select three unanswered probes for targeted study. Repeat the interview after two days with changed assumptions: remote instead of local posting, an older target release, or a direct API consumer instead of Fiori.